

Challenge Algorithmique & Mini-Projet TSP

CHALLENGE : ALGORITHMIQUES REGROUPÉS

Objectif : Développer plusieurs petites fonctions indépendantes, chacune ciblant une compétence précise

Travail à faire :

- Demander à l'utilisateur de saisir un nombre entier `n` et afficher la factorielle de ce nombre (`n!`).
- Demander à l'utilisateur un nombre entier `m` et afficher sa table de multiplication de 1 à 10.
- Demander à l'utilisateur un nombre entier `L` et indiquer s'il s'agit d'un carré parfait.
- Demander une chaîne de caractères à l'utilisateur, puis afficher chaque caractère un par un.
- Demander une phrase à l'utilisateur et afficher le mot le plus long de cette phrase.
- Demander une chaîne de caractères `Ch` et afficher le nombre d'occurrences de chaque caractère.

Exemple : Pour `Ch = "artificial intelligence developer"`

Le programme doit afficher : Le caractère "i" figure 5 fois dans la chaîne Ch.

MINI PROJET : VOYAGEUR DE COMMERCE (TSP)

Sujet de l'examen : Vous êtes chargé de résoudre un problème d'optimisation simplifié inspiré du Voyageur de Commerce (TSP). On vous fournit une liste de villes sous forme de nœuds avec leurs coordonnées (x,y), et vous devez :

1. Charger et représenter les données

On vous donne un fichier texte `villes.txt` contenant sur chaque ligne : `NomVille X Y`

Par exemple :

```
Paris 48.8567 2.3522
Bordeaux 44.84 -0.58
Marseille 43.2964 5.37
...
```

- Écrire une fonction `charger_villes(chemin)` qui prend en paramètre le chemin du fichier (ex : "villes.txt") et retourne une liste de tuples `(NomVille, X, Y)`.
- Afficher la liste des villes chargées ainsi que leur nombre total.

2. Calcul des distances

Vous allez implémenter un calcul de distance Euclidienne entre deux villes.

- Écrire une fonction `distance(villeA, villeB)` qui, étant donné deux tuples `(NomVille, X, Y)`, retourne la distance Euclidienne entre elles.
- Tester votre fonction en affichant la distance entre au moins 2 paires de villes de votre liste.

3. Heuristique simple pour le Voyageur de Commerce

Le but ici n'est pas de résoudre parfaitement le TSP, mais de proposer une heuristique simple : partir de la première ville de la liste, puis à chaque étape, aller à la ville la plus proche non encore visitée. Continuer jusqu'à avoir visité toutes les villes.

- Écrire une fonction `itineraire_greedy(villes)` qui, donnée la liste de villes, retourne une liste représentant l'ordre des visites de ces villes selon l'heuristique décrite (une liste de tuples ou de noms de villes).
- Afficher l'ordre proposé par l'algorithme.

4. Calcul de la distance totale de la tournée

Une fois que vous avez l'itinéraire, vous devez calculer la distance totale parcourue.

- Écrire une fonction `distance_totale(itineraire)` qui calcule la somme des distances entre chaque ville dans l'ordre donné par l'itinéraire, en considérant que l'on ne revient pas à la ville de départ.
- Afficher la distance totale de la tournée retournée par votre heuristique.

5. Affichage et analyse

- Afficher un récapitulatif : nombre total de villes, itinéraire trouvé, distance totale.
- Discuter brièvement (en commentaire dans le code) des limites de cette approche heuristique. Par exemple : pourquoi n'est-elle pas optimale ? Quels sont les problèmes éventuels de cette méthode sur un plus grand ensemble de villes ?